94.7
Pass

Pankaj Khatoria

PDF generated at: 9 Feb 2026 16:11:32 UTC

View this report on HackerRank

Score

94.7% • 180.0 / 190
scored in eClerx - Java Coding Test - 4+ Years in 140 min 7 sec on 9 Feb 2026 19:19:01 IST

Candidate Information

Email	pankaj.khatoria@gmail.com
Test	eClerx - Java Coding Test - 4+ Years
Candidate Packet	View
Taken on	9 Feb 2026 19:19:01 IST
Time taken	140 min 7 sec/ 180 min
Work Experience	> 5 years
Invited by	Vipul
Invited on	7 Feb 2026 13:17:37 IST

Skill Distribution

No.	Skill	Score
1	MongoDB Basic	100%

2	Spring Boot Advanced	50%
3	Spring Boot Intermediate	100%
4	Problem Solving Intermediate	100%
5	Java Intermediate	100%

Tags Distribution

MongoDB	100%	Easy	100%
Insert	100%	NoSQL	100%
Query	100%	Query Parameters	100%
Spring Boot	60%	Hard	50%
TDD	100%	Medium	100%
Sorting	100%	Greedy Algorithms	100%
Two Pointers	100%	Coding	100%
Java	100%	Implementation	100%
Interfaces	100%		

Questions

MongoDB (Basic) • 15 / 15

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	MongoDB: Post Multiple Choice	MongoDB (Basic)	5/5	-	5 min 46 sec	✓
2	MongoDB: Find Multiple Choice	MongoDB (Basic)	5/5	-	2 min 47 sec	✓
3	MongoDB: Find II Multiple Choice	MongoDB (Basic)	5/5	-	5 min 9 sec	✓

Spring Boot (Advanced) • 15 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
4	JSON Deserialization Multiple Choice	Spring Boot (Advanced)	0/5	-	6 min 13 sec	⊖
5	Specification Multiple Choice	Spring Boot (Advanced)	5/5	-	1 min 51 sec	✓
6	Determine the Output Multiple Choice	Spring Boot (Advanced)	0/5	-	3 min 16 sec	✗
7	Response Multiple Choice	Spring Boot (Advanced)	5/5	-	2 min 12 sec	✓

8	CSRF Multiple Choice	Spring Boot (Intermediate)	5/5	-	2 min 9 sec	✓

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
9	Maximum Array Correlation Coding	Problem Solving (Intermediate)	75/75	-	16 min 2 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
10	Java: Library Management System Coding	Java (Intermediate)	75/75	-	1 hour 34 min 4 sec	✓

1. MongoDB: Post

✓ Correct

Multiple Choice | MongoDB | Easy | Insert | NoSQL

Question description

Which of the following queries allows to insert a new document having the name `POST1` into the posts collection?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] db.posts.insert({"name" : "POST1"})
- [ ] db.posts..update({name : 'POST2'}, {name : 'POST1'}, {upsert : true})
- [x] db.posts.save({"name" : "POST1"})
- [ ] db.posts.findOneAndReplace({name : 'POST2'}, {name : 'POST1'}, {upsert : true})

No comments.

2. MongoDB: Find

Correct

Multiple Choice | MongoDB | Easy | NoSQL

Question description

Which of the following queries will return only the amount and _id fields from the documents matching the query?

Interviewer guidelines

Option 3 shows the *_id* by default and the *amount* field.
The *_id* field is included unless there is an explicit statement to omit it, e.g., option 1 with _id: 0.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- db.transactions.find({type : ‘credit’}, {_id : 0, amount : 0})
- db.transactions.find({type : ‘credit’}, {_id : 1, amount : 0})
- **db.transactions.find({type : ‘credit’}, { amount : 1})**
- db.transactions.find({type : ‘credit’}, {amount : 0, type : 1})

No comments.

3. MongoDB: Find II

Correct

Multiple Choice | Easy | Query | Query Parameters

Question description

Consider the following collection of products. Which of the following queries can be used to find only the document having the _id 3?

```
{ _id: 1, item: null, amount : 10 }
{ _id: 2, item: "T-Shirt" }
{ _id: 3}
{ _id: 4, amount: 40}
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- db.products.findOne({amount : {$lt : 10}})
- **db.products.findOne({item : {$exists : false}})**
- b.products.findOne({amount : null })
- db.products.findOne({_id : {$lt : 2, $gt : 4}})

No comments.

4. JSON Deserialization

Not attempted

Multiple Choice | Spring Boot | Hard

Question description

CheckerCreateRequest.java

```
// Annotation here
public class CheckerCreateRequest {
    private String name;
    private String surname;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getSurname() {
        return surname;
    }
```

```
    public void setSurname(String surname) {
        this.surname = surname;
    }
}
```

CheckerController.java

```
@RestController
@RequestMapping("/checkers")
public class CheckerController {

    @Autowired
    CheckerService checkerService;

    @PostMapping
    public ResponseEntity createChecker(@RequestBody CheckerCreateRequest checkerCreateReque
st) {
        checkerService.createChecker(checkerCreateRequest);
        return new ResponseEntity(HttpStatus.CREATED);
    }
}
```

Request Json data

```
{
    "checker":{
        "name": "Checker",
        "surname":"Number2"
    }
}
```

A POST request with "Request Json data" is made to "/checkers". What annotation needs to be added to the "CheckerCreateRequest" class so that it can successfully deserialize and process the request?

Interviewer guidelines

@JsonRootName doesn’t add checker as a wrapper attribute by itself. To make it successfully deserialize with given json annotations on D should be used together.

Candidate's Solution

This candidate has not answered this question.

No comments.

5. Specification

Correct

Multiple Choice | Spring Boot | Hard

Question description

Which of the following options can be used for dynamic jpa queries with spring-data-jpa repositories?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ PreparedStatement
- ● Specification ✓
- ○ Criteria
- ○ Query

No comments.

6. Determine the Output

Incorrect

Multiple Choice | Spring Boot | Hard

Question description

SpringQuestionApplication.java

```
package com.hackerrank.spring;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class SpringQuestionsApplication {
   public static void main(String[] args) {
      SpringApplication.run(SpringQuestionsApplication.class, args);
   }
}
```

Lie.java

```
package com.hackerrank;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
@Component
public class Lie implements CommandLineRunner {
   @Override
   public void run(String... args) throws Exception {
      System.out.println("Truth is behind the lies");
      System.out.println("This is working perfectly");
      throw new Exception("Does this work?");
   }
}
```

Truth.java

```
package com.hackerrank.spring.reality;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class Truth implements CommandLineRunner {
   @Override
```

```
    public void run(String... args) throws Exception {
        System.out.println("Truth is in your eyes");
    }
}
```

What will be printed on the console when the "SpringBootApplication" class is executed?

Interviewer guidelines

Correct Answer:

Option [A] – "Truth is in your eyes"

Explanation: Spring Boot scans only the package of the `@SpringBootApplication` class and its subpackages. `Truth` is under a subpackage and runs. `Lie` is in a parent package, so it is **not scanned** and its runner does not execute.

Incorrect Answers:

Option [B] – Multiple prints + exceptions

Explanation: `Lie` never runs, so its prints and exception do not occur.

Option [C] – Prints from Lie only

Explanation: `Truth` is discovered and executed; `Lie` is not.

Option [D] – Prints from Lie, then Truth

Explanation: `Lie` is outside the scan path and never executed.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- Truth is in your eyes ✓
- Truth is in your eyes Truth is behind the lies This is working perfectly And exception occurs and application stops

Truth is behind the lies This is working perfectly And exception occurs and application stops

Truth is behind the lies This is working perfectly And exception occurs containing “Does this work?” Truth is in your eyes

No comments.

7. Response

Correct

Multiple Choice | Hard | Spring Boot | TDD

Question description

Which response will cause the following test case to fail?

```
@Test
   @DisplayName("Filter Price Check 3")
   public void FilterPriceCheck3() throws Exception {
      mockMvc.perform(get("/filter/price/150/500"))
            .andDo(print())
            .andExpect(jsonPath("$[*].product", containsInAnyOrder("Product 1", "Product 2" , "Produc
t 5")))
   }
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [{"product" : "Product 5"} , {"product" : "Product 1"}, {"product" : "Product 2"}]
- [{"product" : "Product 5"} , {"product" : "Product 2"}, {"product" : "Product 1"}]
- [{"product" : "Product 1"} , {"product" : "Product 2"}, {"product" : "Product 5"}]
- **[{"product" : "product 5"} , {"product" : "product 2"}, {"product" : "product 1"}]**

No comments.

8. CSRF

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which Java Spring-based code segment correctly configures CSRF?

Interviewer guidelines

CSRF Protection makes sure that server is not bombarded with unwanted calls. Spring Security enables some features by default like CSRF and Form based authentication.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- private void manage(HttpSecurity http) {   http.csrf().allowed() }
- private void manage(HttpSecurity http) {   http.csrf() }
- private void manage(HttpSecurity http) {   http.csrf().enable() }
- ** None of the above. CSRF is configured by default when Spring Security Dependency is added. **

No comments.

9. Maximum Array Correlation

Correct

Coding | Sorting | Greedy Algorithms | Medium | Two Pointers

Question description

For two arrays *a* and *b* of equal length, the array correlation is defined as the sum of all values *b[i]* where *b[i]* is greater than *a[i]*.

Given two integer arrays *a* and *b* of the same length, rearrange the elements of array *b* to maximize the array correlation. Return the maximum possible array correlation value.

Note: You are not allowed to rearrange the elements of the array *a*.

Example:

n = 5, *a* = [1, 2, 3, 4, 5], *b* = [3, 5, 4, 6, 2]

b can be rearranged to [2, 3, 4, 5, 6]. Now, for each index, the integers in *b* are greater than those in *a.* The array correlation is 2 + 3 + 4 + 5 + 6 = 20.

Function Description

Complete the function *getMaxArrayCorrelation* in the editor with the following parameter(s):

a[n]: the fixed order array

b[n]: the array to reorder

Returns:

long int: the maximum possible array correlation

Constraints

- $1 \le n \le 2 \times 10^5$
- $1 \le a[i], b[i] \le 10^9$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, that represents the number of elements in *a*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer that represents *a[i].*
The next line contains an integer, *n*, that represents the number of elements in *b*.
Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer that represents *b[i].*

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN       FUNCTION
-------     -----------
5       →  a[] size n = 5
1       →  a = [1, 4, 2, 1, 3]
4
2
1
3
5       →  b[] size n = 5
```

```
2       →  b = [2, 3, 1, 2, 2]
3
1
2
2
```

Sample Output

```
7
```

Explanation

The second array can be rearranged to *b = [2, 1, 3, 2, 2]*. *b[i] > a[i]* for *i =* 0, 3, and 4, and the array correlation is 2 + 3 + 2 = 7.

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
STDIN       FUNCTION
-------     ----------
4       →  a[] size n = 4
1       →  a = [1, 9, 4, 2]
9
4
2
4       →  b[] size n = 4
8       →  b = [8, 4, 3, 1]
4
3
1
```

Sample Output

```
15
```

Explanation

The second array can be rearranged to *b = [3, 1, 8, 4]*. *b[i] > a[i]* for *i =* 0, 3, and, 4. The array correlation is 3 + 8 + 4 = 15.

Interviewer guidelines

▼ SOLUTION

Skills: Greedy Algorithms, Sorting

Optimal Solution:

We can use a greedy approach here. We will always try to take the maximum values of b as much as possible. We can sort the two arrays in reverse order and iterate through the array *a.* We add *b[j]* to the score and increment *j* at each element where *a[i] < b[j].*

Python Solution:

```
def getMaxArrayCorrelation(a, b):
    list.sort(a, reverse = True)
    list.sort(b, reverse = True)
    ans = 0
    i, j = 0, 0
    n = len(a)
    while i < n and j < n:
        if a[i] < b[j]:
            ans += b[j]
            j += 1
        i += 1
    return ans
```

▼ COMPLEXITY ANALYSIS

Time Complexity - O(n logn)

Sorting the arrays takes O(n logn) time in the worst case.

Space Complexity - O(n)

Assuming that the inbuilt sorting functions use a variation of merge sort, sorting uses O(n) extra space.

Candidate's Solution

Language used: **Java 21**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
```

```java
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'getMaxArrayCorrelation' function below.
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY a
     *  2. INTEGER_ARRAY b
     */

    public static long getMaxArrayCorrelation(List<Integer> a, List<Integer> b) {
    // Write your code here
        Collections.sort(a);
        Collections.sort(b);

        int end1 = a.size()-1;
        int end2 = b.size()-1;

        long ans = 0;

        while(end1 >= 0 && end2 >= 0) {
            if(b.get(end2) <= a.get(end1)) {
                end1--;
            } else {
                ans += b.get(end2);
                end1--;
                end2--;
            }
        }
        return ans;
    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
```

```
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int aCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> a = IntStream.range(0, aCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int bCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> b = IntStream.range(0, bCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        long result = Result.getMaxArrayCorrelation(a, b);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED

Testcase 0	Easy	Sample	Success	1	0.0618 sec	37.6 KB
Testcase 1	Easy	Sample	Success	1	0.06 sec	37.5 KB
Testcase 2	Easy	Sample	Success	1	0.0526 sec	37.9 KB
Testcase 3	Easy	Hidden	Success	3	0.0687 sec	37.9 KB
Testcase 4	Easy	Hidden	Success	3	0.0507 sec	37.6 KB
Testcase 5	Medium	Hidden	Success	4	0.0539 sec	37.7 KB
Testcase 6	Medium	Hidden	Success	5	0.0642 sec	39.3 KB
Testcase 7	Medium	Hidden	Success	6	0.0983 sec	41.1 KB
Testcase 8	Medium	Hidden	Success	6	0.0906 sec	41.7 KB
Testcase 9	Hard	Hidden	Success	7	0.1918 sec	65 KB
Testcase 10	Hard	Hidden	Success	7	0.4327 sec	72.6 KB
Testcase 11	Hard	Hidden	Success	7	0.3399 sec	70.5 KB
Testcase 12	Hard	Hidden	Success	7	0.519 sec	91.6 KB

Testcase 13	Hard	Hidden	Success	8	0.8043 sec	152 KB
Testcase 14	Hard	Hidden	Success	9	0.4922 sec	147 KB

No comments.

10. Java: Library Management System

Correct

Coding Medium Coding Java Implementation Interfaces

Question description

Design a library management system with the following requirements:

Book Class

- Create a *Book* class that implements the *IBook* interface
- Include the following properties:
  - *id* (int): Unique identifier for the book
  - *title* (String): Title of the book
  - *author* (String): Author of the book
  - *category* (String): Category or genre of the book
  - *price* (int): Price of the book in currency units

LibrarySystem Class

- Create a *LibrarySystem* class that implements the *ILibrarySystem* interface
- Declare a private instance variable *_books* of type *Map<IBook, Integer>* to store books as keys and their quantities as values

Methods to Implement

Implement the following methods in the *LibrarySystem* class:

- *addBook(IBook book, int quantity)*: Adds the specified book with the given quantity. If the book already exists, update its quantity

- *removeBook(IBook book, int quantity)*: Removes the specified quantity of the book. If quantity becomes zero or negative, remove the book completely
- *calculateTotal()*: Calculates and returns the total value of all books based on their prices and quantities
- *categoryTotalPrice()*: Returns a map with category names as keys and the total price of books in each category as values
- *booksInfo()*: Returns a list of *BooksInfo* objects containing book title, quantity, and price, sorted by title and then quantity
- *categoryAndAuthorWithCount()*: Returns a list of *CategoryAuthorWithCount* objects containing category, author, and total quantity of books by that author in that category, sorted by category and then author

Example

There are 2 Book objects, with i*d, title, author, category, price,* and q*uantity*.

1 PeterPan JamesMatthewBarrie KidsClassics 193 11

2 TheWizardOfOz FrankBaum KidsClassics 394 3

Output

Book Info:

Book Name:PeterPan, Quantity:11, Price:193

Book Name:TheWizardOfOz, Quantity:3, Price:394

Category:KidsClassics, Total Price:3305

Category And Author With Count:

Category:KidsClassics, Author:JamesMatthewBarrie, Count:11

Category:KidsClassics, Author:FrankBaum, Count:3

Total Price: 3305

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer *n*, the number of books.

Each of the next *n* lines contains 6 space-separated values of book attribute values, *id title author category price quantity*.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
4
1 Title-1 Author-1 Category-4 206 14
2 Title-2 Author-2 Category-2 527 23
3 Title-3 Author-2 Category-2 734 6
4 Title-4 Author-1 Category-1 58 29
```

Sample Output

```
Book Info:
Book Name:Title-1, Quantity:14, Price:206
Book Name:Title-2, Quantity:23, Price:527
Book Name:Title-3, Quantity:6, Price:734
Book Name:Title-4, Quantity:29, Price:58
Category Total Price:
Category:Category-1, Total Price:1682
Category:Category-2, Total Price:16525
Category:Category-4, Total Price:2884
Category And Author With Count:
Category:Category-1, Author:Author-1, Count:29
Category:Category-2, Author:Author-2, Count:29
Category:Category-4, Author:Author-1, Count:14
Total Price: 21091
```

Explanation

There are 4 books to add to the list of books. The provided code stub produces the results of calling the methods.

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
3
1 Title-1 Author-1 Category-3 672 4
2 Title-2 Author-2 Category-4 59 29
3 Title-3 Author-1 Category-3 563 29
```

Sample Output

```
Book Info:
Book Name:Title-1, Quantity:4, Price:672
Book Name:Title-2, Quantity:29, Price:59
Book Name:Title-3, Quantity:29, Price:563
Category Total Price:
Category:Category-3, Total Price:19015
Category:Category-4, Total Price:1711
Category And Author With Count:
```

```
Category:Category-3, Author:Author-1, Count:33
Category:Category-4, Author:Author-2, Count:29
Total Price: 20726
```

Explanation

There are 3 books to add to the list. The provided code stub produces the results of calling the methods.

Interviewer guidelines

▼ SOLUTION

Solution

```
class Book implements IBook {
    private int id;
    private String title;
    private String author;
    private String category;
    private int price;

    @Override
    public void setId(int id) {
        this.id = id;
    }

    @Override
    public int getId() {
        return id;
    }

    @Override
    public void setTitle(String title) {
        this.title = title;
    }

    @Override
    public String getTitle() {
        return title;
    }

    @Override
    public void setAuthor(String author) {
        this.author = author;
```

```
    }

    @Override
    public String getAuthor() {
        return author;
    }

    @Override
    public void setCategory(String category) {
        this.category = category;
    }

    @Override
    public String getCategory() {
        return category;
    }

    @Override
    public void setPrice(int price) {
        this.price = price;
    }

    @Override
    public int getPrice() {
        return price;
    }

}

class BookComparator implements Comparator<IBook> {
    public int compare(IBook a, IBook b) {
        return a.getTitle().compareTo(b.getTitle());
    }
}
class LibrarySystem implements ILibrarySystem {
    private final Map<IBook, Integer> books = new TreeMap<IBook, Integer>(new BookComparator
());

    @Override
    public void addBook(IBook book, int quantity) {
        if (books.containsKey(book)) {
            books.put(book, books.get(book) + quantity);
        } else {
            books.put(book, quantity);
        }
    }
```

```
@Override
public void removeBook(IBook book, int quantity) {
    if (books.containsKey(book)) {
        int currentQuantity = books.get(book);
        int newQuantity = currentQuantity - quantity;
        if (newQuantity <= 0) {
            books.remove(book);
        } else {
            books.put(book, newQuantity);
        }
    }
}

@Override
public int calculateTotal() {
    int total = 0;
    for (Map.Entry<IBook, Integer> entry : books.entrySet()) {
        IBook book = entry.getKey();
        int quantity = entry.getValue();
        total += book.getPrice() * quantity;
    }
    return total;
}

@Override
public Map<String, Integer> categoryTotalPrice() {
    Map<String, Integer> categoryPrice = new TreeMap<>();
    for (Map.Entry<IBook, Integer> entry : books.entrySet()) {
        IBook book = entry.getKey();
        int quantity = entry.getValue();
        String category = book.getCategory();
        int totalPrice = book.getPrice() * quantity;
        if(!categoryPrice.containsKey(category))
            categoryPrice.put(category, 0);
        categoryPrice.put(category, categoryPrice.get(category) + totalPrice);
    }
    return categoryPrice;
}

@Override
public List<BooksInfo> booksInfo() {
    List<BooksInfo> list = new ArrayList<>();
    for (Map.Entry<IBook, Integer> entry : books.entrySet()) {
        IBook book = entry.getKey();
        int quantity = entry.getValue();
```

```
            list.add(new BooksInfo(book.getTitle(), quantity, book.getPrice()));
        }
        return list;
    }

    @Override
    public List<CategoryAuthorWithCount> categoryAndAuthorWithCount() {
        Map<String, Map<String, Integer>> categoryAuthorCount = new TreeMap<>();
        for (Map.Entry<IBook, Integer> entry : books.entrySet()) {
            IBook book = entry.getKey();
            int quantity = entry.getValue();
            String category = book.getCategory();
            String author = book.getAuthor();

            if (!categoryAuthorCount.containsKey(category)) {
                categoryAuthorCount.put(category, new TreeMap<String, Integer>());
            }

            Map<String, Integer> authorCount = categoryAuthorCount.get(category);
            if(!authorCount.containsKey(author))
                authorCount.put(author, 0);
            authorCount.put(author, authorCount.get(author) + quantity);
        }

        List<CategoryAuthorWithCount> result = new ArrayList<>();
        for (Map.Entry<String, Map<String, Integer>> categoryEntry : categoryAuthorCount.entrySet()) {
            String category = categoryEntry.getKey();
            Map<String, Integer> authorCount = categoryEntry.getValue();
            for (Map.Entry<String, Integer> authorEntry : authorCount.entrySet()) {
                result.add(new CategoryAuthorWithCount(category, authorEntry.getKey(), authorEntry.get
Value()));
            }
        }

        return result;
    }
}
```

Candidate's Solution

Language used: **Java 21**

```
import java.io.*;
import java.util.*;

interface IBook {
    void setId(int id);
```

```
    int getId();
    void setTitle(String title);
    String getTitle();
    void setAuthor(String author);
    String getAuthor();
    void setCategory(String category);
    String getCategory();
    void setPrice(int price);
    int getPrice();
}

interface ILibrarySystem {
    void addBook(IBook book, int quantity);
    void removeBook(IBook book, int quantity);
    int calculateTotal();
    Map<String, Integer> categoryTotalPrice();
    List<BooksInfo> booksInfo();
    List<CategoryAuthorWithCount> categoryAndAuthorWithCount();
}

class BooksInfo {
    String title;
    int quantity;
    int price;

    BooksInfo(String title, int quantity, int price) {
        this.title = title;
        this.quantity = quantity;
        this.price = price;
    }
}

class CategoryAuthorWithCount {
    String category;
    String author;
    int count;

    CategoryAuthorWithCount(String category, String author, int count) {
        this.category = category;
        this.author = author;
        this.count = count;
    }
}

import java.io.*;
import java.util.*;
```

```
52 import java.util.stream.Collectors;
53
54 interface IBook {
55     void setId(int id);
56     int getId();
57     void setTitle(String title);
58     String getTitle();
59     void setAuthor(String author);
60     String getAuthor();
61     void setCategory(String category);
62     String getCategory();
63     void setPrice(int price);
64     int getPrice();
65 }
66
67 interface ILibrarySystem {
68     void addBook(IBook book, int quantity);
69     void removeBook(IBook book, int quantity);
70     int calculateTotal();
71     Map<String, Integer> categoryTotalPrice();
72     List<BooksInfo> booksInfo();
73     List<CategoryAuthorWithCount> categoryAndAuthorWithCount();
74 }
75
76 class BooksInfo {
77     String title;
78     int quantity;
79     int price;
80
81     BooksInfo(String title, int quantity, int price) {
82         this.title = title;
83         this.quantity = quantity;
84         this.price = price;
85     }
86 }
87
88 class CategoryAuthorWithCount {
89     String category;
90     String author;
91     int count;
92
93     CategoryAuthorWithCount(String category, String author, int count) {
94         this.category = category;
95         this.author = author;
96         this.count = count;
97     }
```

```java

    public String getCategory() {
        return category;
    }

    public String getAuthor() {
        return author;
    }
}

class CategoryAuthorKey {
    public String author;
    public String category;

    public CategoryAuthorKey(String author, String category) {
        this.author = author;
        this.category = category;
    }

    @Override
    public boolean equals(Object o) {
        if(this == o) {
            return true;
        }
        if(o == null) {
            return false;
        }

        CategoryAuthorKey temp = (CategoryAuthorKey) o;
        return Objects.equals(category, temp.category) &&
Objects.equals(author, temp.author);
    }

    @Override
    public int hashCode() {
        return Objects.hash(category, author);
    }
}

class Book implements IBook {
    private int id;
    private String title;
    private String author;
    private String category;
    private int price;

```

```
    public void setId(int id) {
        this.id = id;
    }

    public int getId() {
        return this.id;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getTitle() {
        return this.title;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public String getAuthor() {
        return this.author;
    }

    public void setPrice(int price) {
        this.price = price;
    }

    public int getPrice() {
        return this.price;
    }

    public void setCategory(String category) {
        this.category = category;
    }

    public String getCategory() {
        return this.category;
    }

    @Override
    public boolean equals(Object object) {
        if(this == object) {
            return true;
        }
        if(object == null) {
```

```
            return false;
        }
        Book b = (Book) object;
        return b.id == id;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}

class LibrarySystem implements ILibrarySystem {
    private final HashMap<IBook, Integer> _books;

    public LibrarySystem() {
        this._books = new HashMap<>();
    }

    public void addBook(IBook book, int quantity) {
        if(_books.containsKey(book)) {
            int prevQty = _books.get(book);
            _books.put(book, prevQty+quantity);
        } else {
            _books.put(book, quantity);
        }
    }

    public void removeBook(IBook book, int quantity) {
         if(!_books.containsKey(book)) {
            return;
         }
         int prevQty = _books.get(book);
         int updatedQty = prevQty - quantity;
         if(updatedQty <= 0) {
            _books.remove(book);
            return;
         }
         _books.put(book, updatedQty);
    }

    public int calculateTotal() {
        int total = 0;

        for(Map.Entry<IBook, Integer> entry :  _books.entrySet()) {
            total += entry.getValue() * entry.getKey().getPrice();
```

```java
        }
        return total;
    }

    public Map<String, Integer> categoryTotalPrice() {

        HashMap<String, Integer> ans = new HashMap<>();
        for(Map.Entry<IBook, Integer> entry :  _books.entrySet()) {
            String cat = entry.getKey().getCategory();
            Integer catPrice = entry.getValue() * entry.getKey().getPrice();

            if(ans.containsKey(cat)) {
                ans.put(cat, ans.get(cat) + catPrice);
            } else {
                ans.put(cat, catPrice);
            }
        }
        return new TreeMap<>(ans);
    }

    public List<BooksInfo> booksInfo() {
        List<BooksInfo> ans = new ArrayList<>();

         for(Map.Entry<IBook, Integer> entry :  _books.entrySet()) {
            BooksInfo bi = new BooksInfo(entry.getKey().getTitle(), entry.getValue(), entry.getKey().getPrice());
            ans.add(bi);
        }

        Collections.sort(ans, new Comparator<BooksInfo>() {

            @Override
            public int compare(BooksInfo b1, BooksInfo b2) {
                int temp = b1.title.compareTo(b2.title);
                if(temp != 0) {
                    return temp;
                }
                return Integer.compare(b1.quantity, b2.quantity);
            }
        });

        return ans;
    }

    public List<CategoryAuthorWithCount> categoryAndAuthorWithCount() {
        return
```

```
        _books.entrySet()
        .stream()
        .collect(
            Collectors.groupingBy(k -> new CategoryAuthorKey(k.getKey().getAuthor(), k.getKey().getCategory()),
            Collectors.summingInt(Map.Entry::getValue))
        )
        .entrySet()
        .stream()
        .map(o -> new CategoryAuthorWithCount(
            o.getKey().category,
            o.getKey().author,
            o.getValue()
        ))
        .sorted(
            Comparator.comparing(CategoryAuthorWithCount::getCategory)
            .thenComparing(CategoryAuthorWithCount::getAuthor)
        ).collect(Collectors.toList());
    }

}

public class Solution {
    public static void main(String[] args) throws IOException{
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(System.out);

        ILibrarySystem librarySystem = new LibrarySystem();
        int bCount = Integer.parseInt(br.readLine().trim());
        for (int i = 1; i <= bCount; i++) {
            String[] a = br.readLine().trim().split(" ");
            IBook e = new Book();
            e.setId(Integer.parseInt(a[0]));
            e.setTitle(a[1]);
            e.setAuthor(a[2]);
            e.setCategory(a[3]);
            e.setPrice(Integer.parseInt(a[4]));

            librarySystem.addBook(e, Integer.parseInt(a[5]));
        }

        out.println("Book Info:");
        List<BooksInfo> booksInfo = librarySystem.booksInfo();
        for (BooksInfo entry : booksInfo) {
            String title = entry.title;
```

```
            int quantity = entry.quantity;
            int price = entry.price;
            out.println("Book Name:" + title + ", Quantity:" + quantity + ", Price:" + price);
        }

        out.println("Category Total Price:");
        Map<String, Integer> categoryTotalPrice = librarySystem.categoryTotalPrice();
        for (Map.Entry<String, Integer> entry : categoryTotalPrice.entrySet()) {
            String category = entry.getKey();
            int totalPrice = entry.getValue();
            out.println("Category:" + category + ", Total Price:" + totalPrice);
        }

        List<CategoryAuthorWithCount> categoryAndAuthorWithCount = librarySystem.categoryAndAuthorWithCount();
        out.println("Category And Author With Count:");
        for (CategoryAuthorWithCount entry : categoryAndAuthorWithCount) {
            String category = entry.category;
            String author = entry.author;
            int count = entry.count;
            out.println("Category:" + category + ", Author:" + author + ", Count:" + count);
        }

        int total = librarySystem.calculateTotal();
        out.println("Total Price: " + total);

        out.flush();
        out.close();
    }
}

public class Solution {
    public static void main(String[] args) throws IOException{
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(System.out);

        ILibrarySystem librarySystem = new LibrarySystem();
        int bCount = Integer.parseInt(br.readLine().trim());
        for (int i = 1; i <= bCount; i++) {
            String[] a = br.readLine().trim().split(" ");
```

```
            IBook e = new Book();
            e.setId(Integer.parseInt(a[0]));
            e.setTitle(a[1]);
            e.setAuthor(a[2]);
            e.setCategory(a[3]);
            e.setPrice(Integer.parseInt(a[4]));

            librarySystem.addBook(e, Integer.parseInt(a[5]));
        }

        out.println("Book Info:");
        List<BooksInfo> booksInfo = librarySystem.booksInfo();
        for (BooksInfo entry : booksInfo) {
            String title = entry.title;
            int quantity = entry.quantity;
            int price = entry.price;
            out.println("Book Name:" + title + ", Quantity:" + quantity + ", Price:" + price);
        }

        out.println("Category Total Price:");
        Map<String, Integer> categoryTotalPrice = librarySystem.categoryTotalPrice();
        for (Map.Entry<String, Integer> entry : categoryTotalPrice.entrySet()) {
            String category = entry.getKey();
            int totalPrice = entry.getValue();
            out.println("Category:" + category + ", Total Price:" + totalPrice);
        }

        List<CategoryAuthorWithCount> categoryAndAuthorWithCount = librarySystem.categoryAndAuthorWithCount();
        out.println("Category And Author With Count:");
        for (CategoryAuthorWithCount entry : categoryAndAuthorWithCount) {
            String category = entry.category;
            String author = entry.author;
            int count = entry.count;
            out.println("Category:" + category + ", Author:" + author + ", Count:" + count);
        }

        int total = librarySystem.calculateTotal();
        out.println("Total Price: " + total);

        out.flush();
```

```
        out.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample	Success	1	0.097 sec	41.5 KB
Testcase 1	Easy	Sample	Success	1	0.0928 sec	41.3 KB
Testcase 2	Easy	Sample	Success	1	0.0844 sec	41.1 KB
Testcase 3	Easy	Hidden	Success	1	0.0853 sec	41.5 KB
Testcase 4	Easy	Hidden	Success	1	0.0841 sec	41.6 KB
Testcase 5	Medium	Hidden	Success	10	0.1019 sec	42.1 KB
Testcase 6	Medium	Hidden	Success	7	0.0986 sec	41.9 KB
Testcase 7	Medium	Hidden	Success	10	0.1244 sec	41.9 KB
Testcase 8	Medium	Hidden	Success	6	0.11 sec	41.7 KB
Testcase 9	Medium	Hidden	Success	7	0.0938 sec	42 KB
Testcase 10	Hard	Hidden	Success	10	0.1666 sec	47.9 KB

Testcase 11	Hard	Hidden	Success	10	0.1789 sec	47.1 KB
Testcase 12	Hard	Hidden	Success	10	0.1327 sec	45 KB

No comments.